

Contents

1. Introduction to Trees and Networks and Syllabus	2
2. Trees — Basic Properties	3
2.1 Definition · 2.2 Characterization · 2.3 Counting edges	3
3. Rooted Trees, Levels and Terminology	5
4. Binary Trees and Full Binary Trees	6
5. Spanning Trees	7
6. Depth-First Search (DFS)	8
7. Breadth-First Search (BFS)	9
8. Weighted Graphs and Minimum Spanning Trees	10
9. Kruskal's Algorithm	11
10. Prim's Algorithm	12
11. Shortest Path Problem — Dijkstra's Algorithm	13
12. Network Flow Problem	14
13. Residual Networks and Augmenting Paths	15
14. Ford-Fulkerson Algorithm	16
15. Max-Flow Min-Cut Theorem	17
16. Question Bank — Unit IV (Fully Solved)	18
17. Quick Revision Summary	24

1. Introduction to Trees and Networks and Syllabus

A **tree** is the simplest kind of connected graph: it has no cycles at all. Trees are everywhere — file systems, family trees, organisation charts, expression trees in compilers, and decision trees in machine learning are all trees. Because a tree has exactly one route between any two vertices, it is the most economical way to keep a set of items connected.

A **network** (flow network) is a directed graph in which every edge carries a *capacity*, and we ask how much "material" (water, data, traffic, goods) can be pushed from a source to a sink. This single model answers questions in transport, communications and logistics.

1.1 Syllabus of Unit IV (Trees and Networks)

- Trees — basic properties — **Sections 2-4**
- Spanning trees — **Section 5**
- Depth-first search (DFS) and breadth-first search (BFS) — **Sections 6-7**
- Minimal spanning tree; Kruskal's and Prim's algorithms — **Sections 8-10**
- Shortest path problem; Dijkstra's algorithm — **Section 11**
- Network flow problem; Ford-Fulkerson algorithm; Max-Flow Min-Cut theorem — **Sections 12-15**
- Solved Question Bank — **Section 16**

2. Trees — Basic Properties

Definition (Tree). A **tree** is a connected undirected graph with **no simple circuits** (no cycles). Equivalently: a tree is a simple graph in which for every pair of vertices v and w there is a **unique** path from v to w .

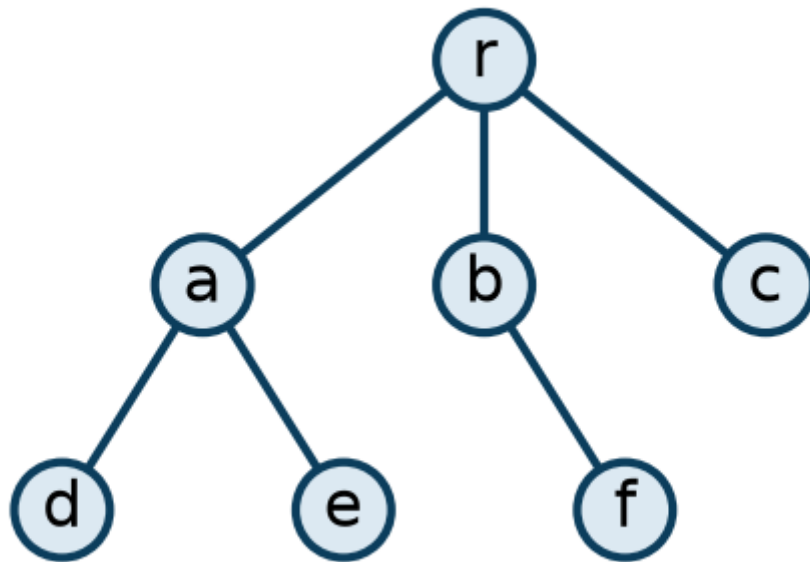
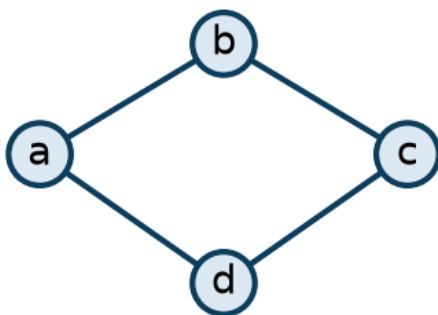


Fig. 2.1 — A tree on 7 vertices with 6 edges. It is connected and has no cycle.

Tree vs. non-tree. A graph is **not** a tree if it is disconnected, or if it contains a cycle (a circuit), or both. In Fig. 2.2 the left graph is a tree; the right graph has the cycle $a-b-c-a$ and so is not a tree.

Tree (no cycle)



Not a tree (has cycle)

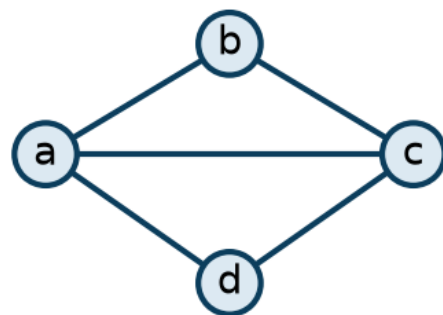


Fig. 2.2 — Left: a tree. Right: not a tree, because of the cycle.

2.1 Characterization of Trees

Theorem (Equivalent conditions). If T is a graph with n vertices, the following are **all equivalent**:

1. T is a tree;
2. T is connected and acyclic (has no cycles);
3. T is connected and has exactly $n-1$ edges;
4. T is acyclic and has exactly $n-1$ edges.

Why $n-1$ edges? Start with the n isolated vertices (0 edges). Joining two vertices that are not yet connected by a new edge keeps the graph acyclic and reduces the number of components by 1. To go from n components down to 1 (connected) we must add exactly $n-1$ such edges. Adding any further edge would have to join two vertices already in the same component — creating a cycle. So a tree has exactly $n-1$ edges. $\square$

Useful facts.

- A tree with n vertices has exactly $n-1$ edges.
- Adding any one edge to a tree creates exactly one cycle.
- Removing any one edge from a tree disconnects it (every edge is a bridge).
- A tree with $n \geq 2$ vertices has at least two vertices of degree 1 (leaves).

3. Rooted Trees, Levels and Terminology

Rooted Tree. A **rooted tree** is a tree in which one vertex is designated as the **root**. The root sits at the top, and edges are thought of as directed away from it.

Level and Height. Let T be a rooted tree.

- The **level** $\ell(v)$ of a vertex v is the length of the simple path from v to the root.
- The **height** h of the tree is the maximum of all the level numbers: $h = \max_v \ell(v)$.

Rooted tree (root r , height 2)

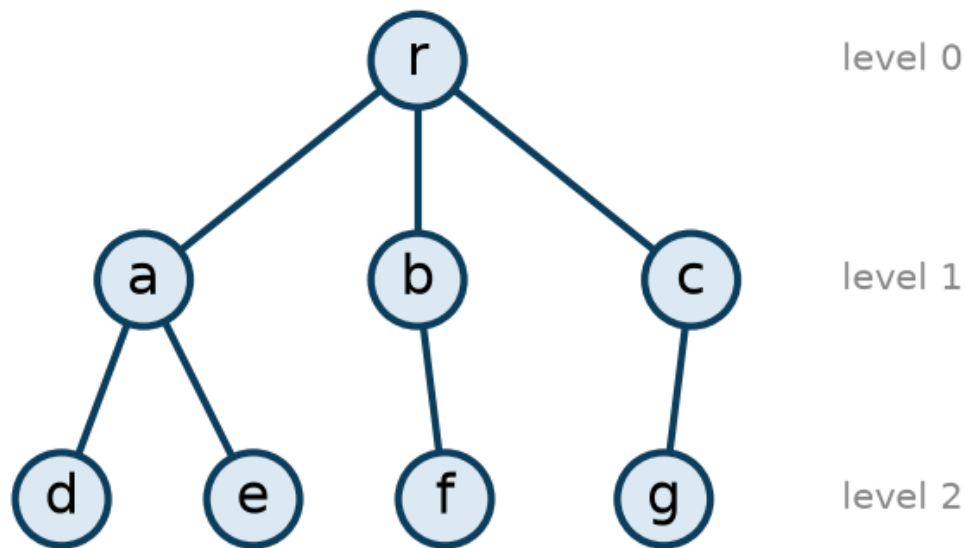


Fig. 3.1 — A rooted tree. The root r is at level 0, its children at level 1, and their children at level 2; the height is 2.

3.1 Family Terminology

- **Parent / Child:** if there is an edge from u down to v , then u is the parent of v and v is a child of u .
- **Siblings:** vertices with the same parent.
- **Ancestor / Descendant:** the vertices on the path from the root down to a vertex are its ancestors; vertices below it are its descendants.
- **Internal vertex:** a vertex that has at least one child.
- **Terminal vertex (leaf):** a vertex with no children.
- **Subtree:** a vertex together with all its descendants forms a subtree.

In Fig. 3.1, the root is r ; its children are a , b , c ; the internal vertices are r , a , b , c and the terminal vertices (leaves) are d , e , f , g .

4. Binary Trees and Full Binary Trees

Binary Tree. A **binary tree** is a rooted tree in which every vertex has **zero, one, or two** children.

Full Binary Tree. A **full binary tree** is a binary tree in which every vertex has either **two children or none** (no vertex has exactly one child).

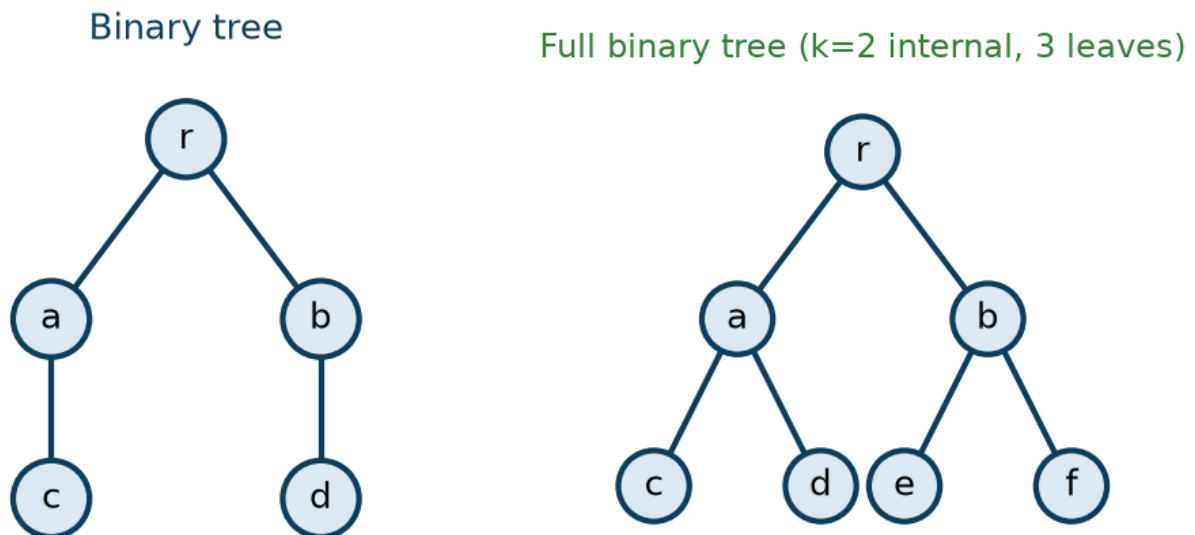


Fig. 4.1 — Left: a binary tree. Right: a full binary tree (every internal vertex has two children).

Theorem (Full binary trees). If T is a full binary tree with k internal vertices, then

- T has $k+1$ terminal vertices (leaves), and
- the total number of vertices is $2k+1$.

Idea. A full binary tree with k internal vertices has exactly $2k$ edges (each internal vertex has 2 children). Since a tree with V vertices has $V-1$ edges, we get $V-1 = 2k$, so $V = 2k+1$. The terminal vertices are the non-internal ones: $V - k = (2k+1) - k = k+1$. $\square$

Theorem (Height and leaves). If a binary tree of height h has t terminal vertices, then $\log_2 t \leq h$, $t \leq 2^h$.

Equality case. If every leaf of a full binary tree is at the same level h (the height), then $t = 2^h$ — the tree is "perfectly balanced".

5. Spanning Trees

Spanning Tree. Given a connected graph G , a tree T is a **spanning tree** of G if T is a subgraph of G and T contains **all the vertices** of G .

So a spanning tree keeps every vertex but throws away enough edges to destroy all cycles. Since it has all n vertices and is a tree, it has exactly $n-1$ edges.

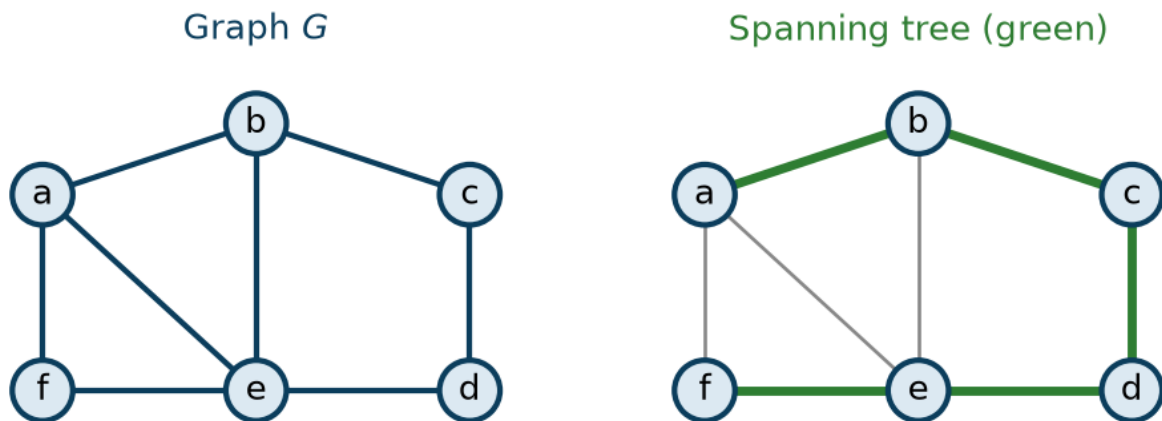


Fig. 5.1 — Left: the graph G . Right: one spanning tree of G (green), containing all 6 vertices and 5 edges.

Example. A graph with 4 vertices and 5 edges has spanning trees each with $4-1=3$ edges; we get them by deleting 2 of the 5 edges (without disconnecting the graph). A Hamiltonian path of a graph is always a spanning tree, because it is connected, contains all vertices and has $n-1$ edges.

How to build one. Any traversal (DFS or BFS, below) that records the edges used to reach a new vertex produces a spanning tree. This is why DFS and BFS are the standard tools for spanning trees.

6. Depth-First Search (DFS)

DFS explores a graph by going **as deep as possible** along one branch before backtracking. It naturally produces a spanning tree of a connected graph.

DFS algorithm (producing a spanning tree).

1. Choose a starting (root) vertex r ; mark it visited.
2. From the current vertex, move to any **unvisited** neighbour; mark it visited and add the edge to the tree.
3. Repeat step 2 from the new vertex (go deeper).
4. When you reach a vertex with no unvisited neighbours, **backtrack** to the previous vertex and continue.
5. Stop when every vertex has been visited; the recorded edges form a spanning tree.

DFS from a : a, b, c, d, e, f

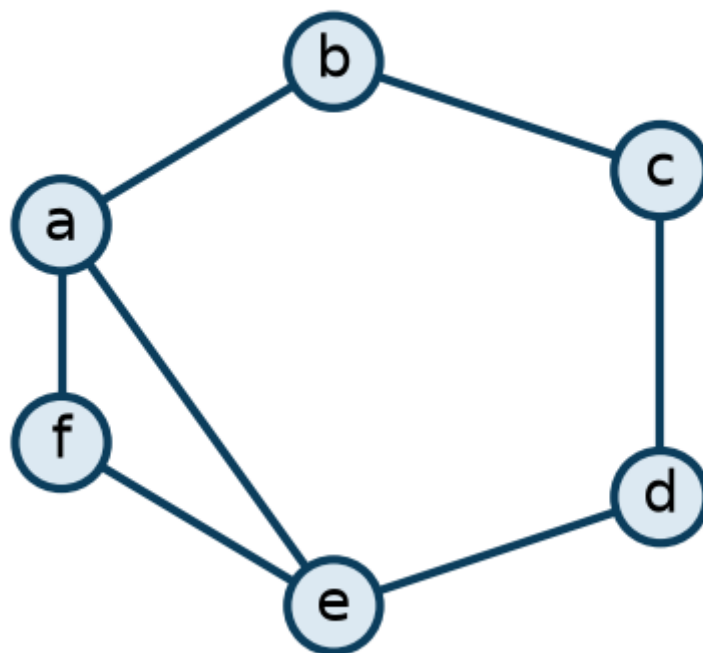


Fig. 6.1 — DFS starting at a : visit order a, b, c, d, e, f (go deep along $a-b-c-d-e$, then backtrack).

Example (Fig. 6.1). Start at a . Go to b , then c , then d , then e , then f . At f we backtrack, but all vertices are already visited, so the DFS tree is the path $a-b-c-d-e-f$ plus the edge $f-a$ — here the tree is $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow f$. DFS uses a **stack** (or recursion).

7. Breadth-First Search (BFS)

BFS explores a graph **level by level**: it visits all neighbours of the current vertex before going deeper. It also produces a spanning tree (the "BFS tree"), whose levels equal the graph distance from the root.

BFS algorithm.

1. Choose a starting vertex r ; mark it visited and put it in a **queue**.
2. Remove the front vertex of the queue and look at all its **unvisited** neighbours.
3. Mark each such neighbour visited, add the edge to the tree, and put it at the back of the queue.
4. Repeat until the queue is empty.

BFS from a : a, b, f, e, c, d

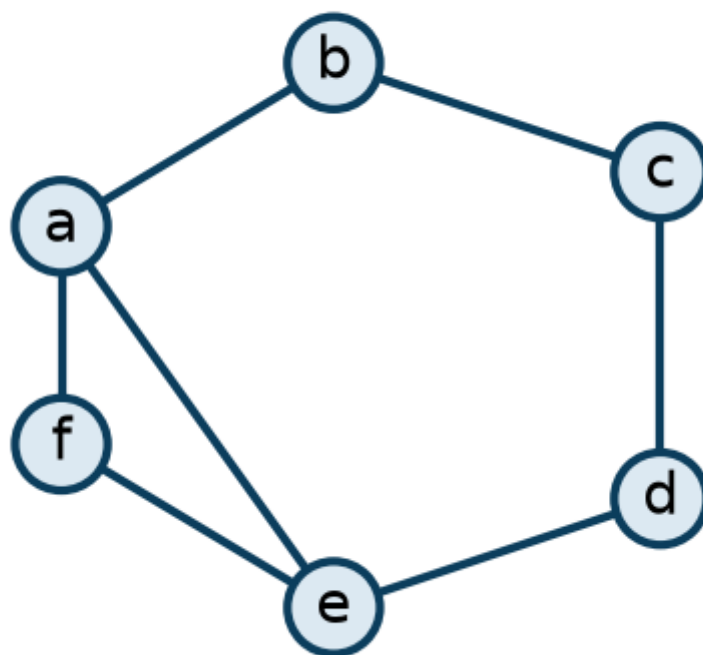


Fig. 7.1 — BFS starting at a : visit order a, b, f, e, c, d — all neighbours of a first, then the next level.

Example (Fig. 7.1). Start at a (level 0). Its neighbours b, f are level 1. Then the unvisited neighbours of b and f are level 2, and so on. BFS uses a **queue** (first-in-first-out).

DFS vs BFS. DFS goes deep (stack); BFS goes wide (queue). Both visit every vertex of a connected graph and both build a spanning tree. BFS is used for shortest paths in *unweighted* graphs; Dijkstra (Section 11) generalises this to weighted graphs.

8. Weighted Graphs and Minimum Spanning Trees

Weighted Graph. A **weighted graph** is a graph in which each edge e is assigned a non-negative number $w(e)$, called its **weight** (think distance, cost or time).

Minimum Spanning Tree (MST). Let G be a connected weighted graph. A **minimum spanning tree** is a spanning tree of G whose total weight (the sum of the weights of its edges) is as small as possible.

Weighted graph

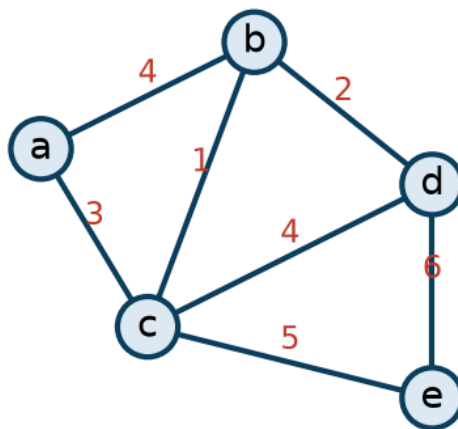


Fig. 8.1 — A weighted graph.

Applications. Laying cables or pipelines to connect all cities with minimum total length, designing least-cost road or network links, clustering, and image segmentation.

Two classical greedy algorithms find an MST: **Kruskal's** (add cheapest edges that do not form a cycle) and **Prim's** (grow one tree from a start vertex). Both are explained next.

9. Kruskal's Algorithm

Kruskal's algorithm is **greedy on edges**: it considers edges from cheapest to most expensive, taking an edge whenever it does not create a cycle.

Kruskal's algorithm.

1. List all edges of G in order of **non-decreasing weight**.
2. Select an edge of minimum weight that is not a loop.
3. At each successive step, select the remaining edge of minimum weight that does **not form a circuit** with the edges already selected.
4. Continue until $n-1$ edges have been selected — these form the MST.

Minimum spanning tree (weight 11)

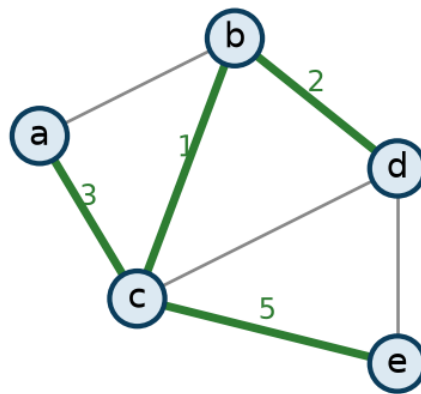


Fig. 9.1 — The MST of Fig. 8.1 (green), total weight $1+2+3+5=11$.

Worked example (Fig. 8.1). Edges by weight: $bc(1)$, $bd(2)$, $ac(3)$, $ab(4)$, $cd(4)$, $ce(5)$, $de(6)$.

- Take $bc(1)$.
- Take $bd(2)$ (no cycle).
- Take $ac(3)$ (no cycle) — now $\{a,b,c,d\}$ connected.
- Skip $ab(4)$ and $cd(4)$ (each would form a cycle).
- Take $ce(5)$, connecting e .

We now have $n-1 = 4$ edges: bc , bd , ac , ce .

MST weight = $1 + 2 + 3 + 5 = 11$

10. Prim's Algorithm

Prim's algorithm is **greedy on vertices**: it grows a single tree, one vertex at a time, always attaching the nearest vertex not yet in the tree.

Prim's algorithm.

1. Draw the n isolated vertices $v_1, v_2, \dots, v_n$.
2. Write the edge weights in an $n \times n$ table (use a very large number for missing edges).
3. Start from v_1 and connect it to its nearest neighbour (smallest entry in row 1), say v_k .
4. Treat $\{v_1, v_k\}$ as one subgraph and connect it to its nearest neighbour outside it (smallest entry in rows 1 and k), say v_i .
5. Continue, each time adding the cheapest edge that joins the current tree to a new vertex, until all n vertices are connected by $n-1$ edges.

Worked example (Fig. 8.1). Start at a : nearest neighbour is c (weight 3) $\rightarrow$ take ac . Tree $\{a,c\}$: cheapest edge out is cb (1) $\rightarrow$ take bc . Tree $\{a,b,c\}$: cheapest edge out is bd (2) $\rightarrow$ take bd . Tree $\{a,b,c,d\}$: cheapest edge out is ce (5) $\rightarrow$ take ce .

MST = ac, bc, bd, ce ; weight 11 — the same tree Kruskal found.

Kruskal vs Prim. Both give an MST (an MST need not be unique). Kruskal builds a forest edge-by-edge; Prim grows one tree vertex-by-vertex. Kruskal is convenient on sparse graphs; Prim is convenient on dense graphs.

11. Shortest Path Problem — Dijkstra's Algorithm

The shortest-path problem asks: given a weighted graph and a source vertex, what is the least-weight path to every other vertex? **Dijkstra's algorithm** answers this for graphs with **non-negative** edge weights.

Idea. The algorithm keeps a label $D[u]$ for each vertex u — the best known distance from the source. It repeatedly "settles" the unsettled vertex with the smallest label (its distance is then final) and relaxes its neighbours.

Dijkstra's algorithm.

1. Set $D(\text{source}) = 0$ and $D(v) = \infty$ for every other vertex.
2. Let the "cloud" be the set of settled vertices (initially empty).
3. Pick the unsettled vertex u with the smallest $D[u]$; add it to the cloud.
4. For each neighbour n of u not in the cloud, **relax**: if $D[u] + w(u,n) < D[n]$, set $D[n] = D[u] + w(u,n)$ (and remember u as n 's predecessor).
5. Repeat steps 3–4 until all vertices are settled.

The shortest path is recovered by following the predecessor pointers back from the target.

Dijkstra: shortest $s \rightarrow t = 6$ ($s \rightarrow b \rightarrow d \rightarrow t$)

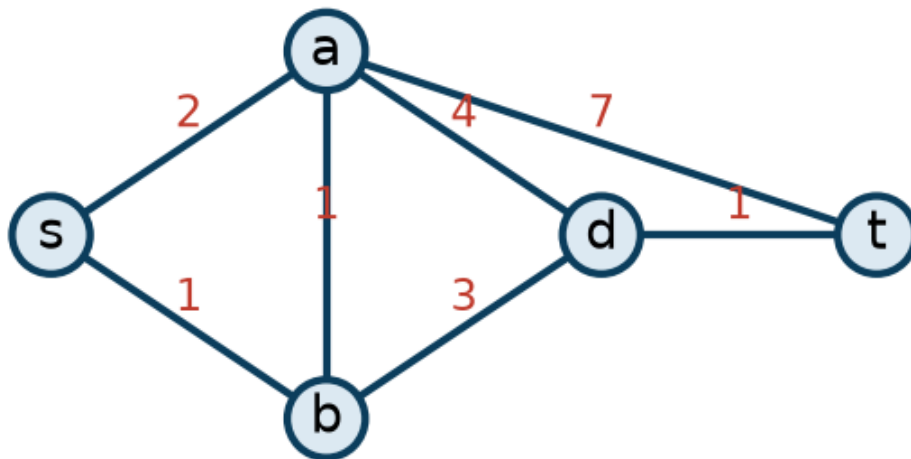


Fig. 11.1 — Shortest path from s to t is $s \rightarrow b \rightarrow d \rightarrow t$ with length $1+3+1 = 6$.

Conditions. Dijkstra works only for **connected** graphs and only when all edge weights are **non-negative**. It works for directed and undirected graphs. The plain algorithm gives the distances; a small modification (keeping predecessors) gives the actual paths.

12. Network Flow Problem

A flow network models a situation where something must be moved from a **source** to a **sink** through pipes of limited capacity.

Flow Network. A **flow network** $G = (V, E)$ is a directed graph in which each edge (u, v) has a non-negative **capacity** $c(u, v) \geq 0$. Two special vertices are distinguished: the **source** s (produces material) and the **sink** t (consumes it).

Flow. A **flow** is a function $f(u, v)$ on the edges satisfying:

- **Capacity constraint:** $0 \leq f(u, v) \leq c(u, v)$ — you cannot push more than the pipe allows;
- **Flow conservation:** for every vertex u s, t , $\sum_{\{v\}} f(u, v) = 0$ — whatever flows in must flow out (nothing is created or lost at an intermediate vertex).

The **value of the flow** is $|f| = \sum_{\{v\}} f(s, v)$, the total net flow leaving the source.

Flow network (edge labels = capacities)

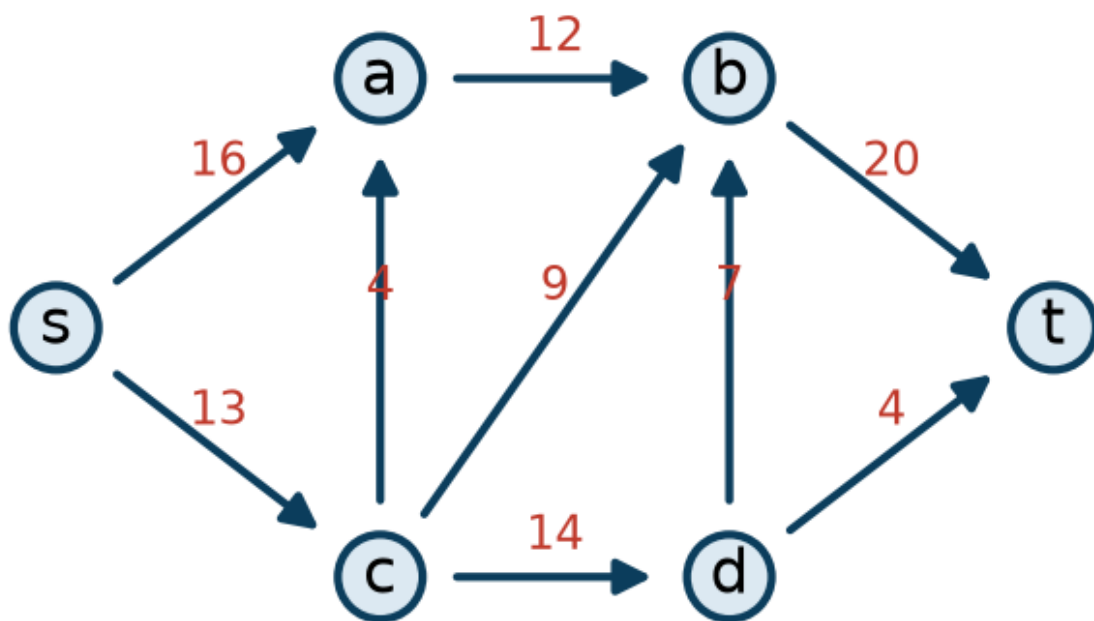


Fig. 12.1 — A flow network; each edge is labelled with its capacity.

Objective. Find a flow of **maximum value** from s to t — the **maximum-flow problem**.

13. Residual Networks and Augmenting Paths

Residual Capacity. Given a flow f , the **residual capacity** of an edge (u,v) is the extra flow we can still push along it: $c_f(u,v) = c(u,v) - f(u,v) \geq 0$. For example, if $c(u,v)=16$ and $f(u,v)=5$, then $c_f(u,v)=11$.

Residual Network. The **residual network** G_f has the same vertices, and an edge (u,v) whenever $c_f(u,v) > 0$. Each such edge can carry a positive additional flow. (If we push flow along (u,v) , we may later "push it back" along the reverse edge (v,u) — this is how the algorithm can undo a bad choice.)

Augmenting Path. An **augmenting path** is a path from s to t in the residual network along which we can push more flow. Its **bottleneck** is the smallest residual capacity on the path; we can push exactly that much additional flow.

Why residual networks matter. If no augmenting path exists in the residual network, the current flow is already maximum. This is the engine behind the Ford-Fulkerson method and the max-flow min-cut theorem.

14. Ford-Fulkerson Algorithm

The **Ford-Fulkerson method** finds a maximum flow by repeatedly finding an augmenting path and pushing flow along it, until no augmenting path remains.

Ford-Fulkerson algorithm.

1. Start with the zero flow: $f(u,v) = 0$ for every edge.
2. Build the residual network G_f .
3. Find an augmenting path P from s to t in G_f . If none exists, **stop** — the current flow is maximum.
4. Let b = the bottleneck of P (the minimum residual capacity along P). For each edge on P , increase the flow by b (and decrease the flow on the reverse edge).
5. Go back to step 2.

Example (Fig. 12.1). Augmenting the paths step by step gives flows: $s \rightarrow a \rightarrow b \rightarrow t$ (11), $s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$ (7), $s \rightarrow c \rightarrow d \rightarrow t$ (4), $s \rightarrow c \rightarrow b \rightarrow t$ (2). The total into t is $20 + 4 = 24$, and no augmenting path remains.

Maximum flow = 24 (labels = flow/capacity)

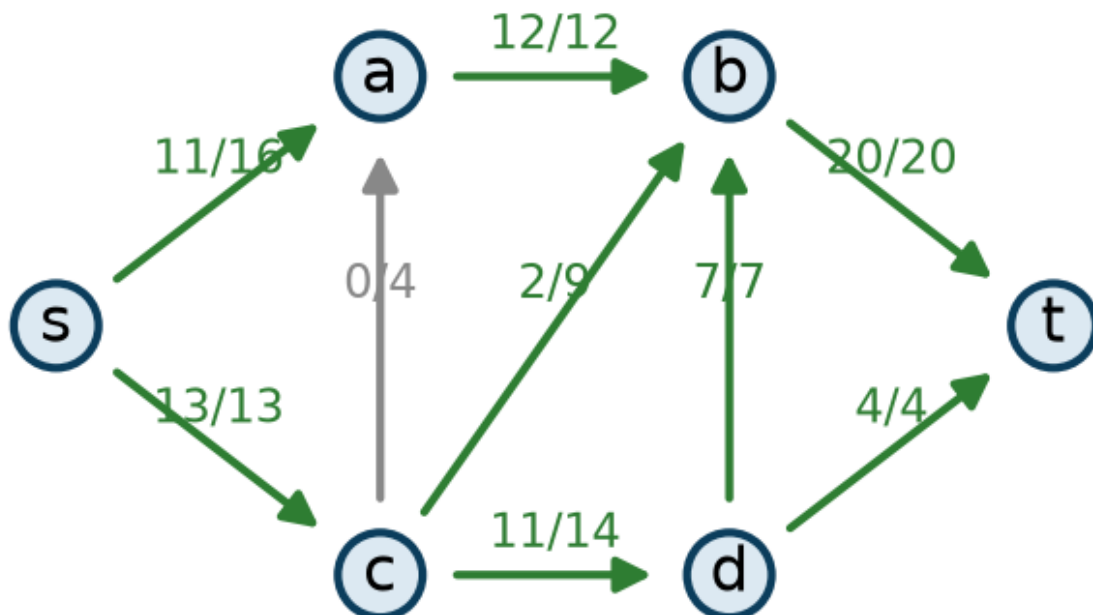


Fig. 14.1 — The maximum flow (labels are flow/capacity). Total flow value = 24.

Why not just guess? A trial-and-error choice of paths may get stuck below the true maximum. Ford-Fulkerson, by always working on the residual network (and allowing flow to be pushed back), **guarantees** the optimum.

15. Max-Flow Min-Cut Theorem

Cut. An **s-t cut** is a partition of the vertices into two sets S (containing s) and T (containing t). Its **capacity** is the sum of the capacities of the edges going from S to T : $\text{cap}(S, T) = \sum_{\{u \in S, v \in T\}} c(u, v)$. A **minimum cut** is an s-t cut of smallest capacity.

Max-Flow Min-Cut Theorem. In any flow network, the value of a **maximum flow** equals the capacity of a **minimum cut**: $\max |f| = \min_{(S, T)} \text{cap}(S, T)$.

Idea of the proof. Every flow must cross any cut, so the flow value can never exceed any cut's capacity — hence $\max \text{flow} \leq \min \text{cut}$. Conversely, when Ford-Fulkerson stops, there is no augmenting path; the set S of vertices reachable from s in the residual network defines a cut whose capacity equals the current flow value — hence $\max \text{flow} \geq \min \text{cut}$. Together they give equality. $\square$

Example (Fig. 12.1). Take $S = \{s, a, b, c, d\}$ and $T = \{t\}$. The edges from S to T are $b \rightarrow t$ (20) and $d \rightarrow t$ (4), so $\text{cap} = 24$. This equals the maximum flow — so this is a minimum cut.

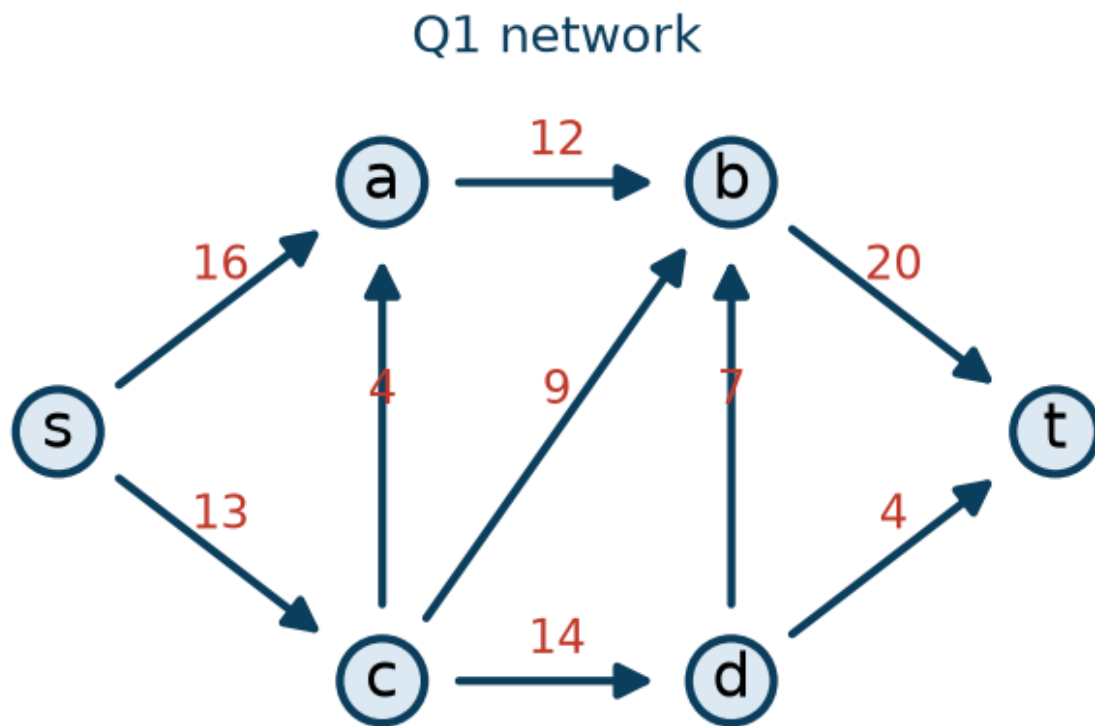
Why it is powerful. It turns the "how much can we push?" question into the "where is the cheapest bottleneck?" question. It also proves that the maximum flow is an integer whenever all capacities are integers.

16. Question Bank — Unit IV (Fully Solved)

Each solution states what is asked, the idea/tool used and why, the steps, and the final boxed answer.

Q1. Find the maximum flow in the transport network shown, and determine the corresponding minimum cut.

What is asked: the largest flow from s to t, and a cut achieving it.



Q1 network — edge labels are capacities.

Method. Use Ford-Fulkerson: keep pushing flow along augmenting paths until none remains. Here the capacities are $s \rightarrow a: 16$, $s \rightarrow c: 13$, $c \rightarrow a: 4$, $a \rightarrow b: 12$, $c \rightarrow b: 9$, $c \rightarrow d: 14$, $d \rightarrow b: 7$, $b \rightarrow t: 20$, $d \rightarrow t: 4$.

Max flow. The two edges into t are $b \rightarrow t$ (capacity 20) and $d \rightarrow t$ (capacity 4). We can saturate both, giving a flow of $20+4=24$ (all of $s \rightarrow a$, $a \rightarrow b$, $b \rightarrow t$ and $d \rightarrow t$ carry their full capacity).

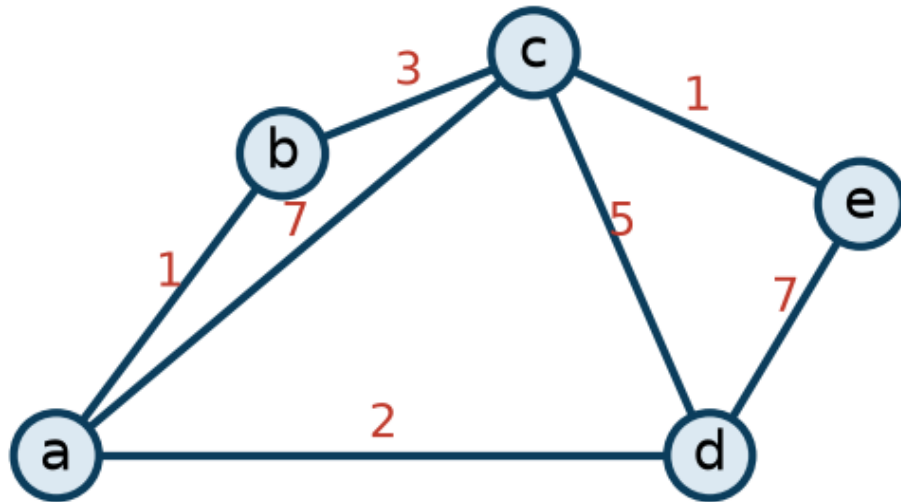
Minimum cut. Take $S = \{s, a, b, c, d\}$ and $T = \{t\}$. The edges from S to T are $b \rightarrow t$ (20) and $d \rightarrow t$ (4), so $\text{cap}(S, T) = 24$ — equal to the max flow.

Maximum flow = 24; minimum cut = $\{b \rightarrow t, d \rightarrow t\}$, capacity 24

Q2. Find the shortest distance and shortest path from a to e using Dijkstra's algorithm.

What is asked: the least-weight route from a to e.

Q2 graph



Q2 graph.

Run Dijkstra from a.

- $a=0$; neighbours: $b=1$, $c=7$, $d=2$.
- Settle b (1): relax c via b: $1+3=4 < 7 \rightarrow c=4$.
- Settle d (2): via d, c would be $2+5=7 > 4$ (no change); e via d: $2+7=9$.
- Settle c (4): relax e via c: $4+1=5 < 9 \rightarrow e=5$.
- Settle e (5).

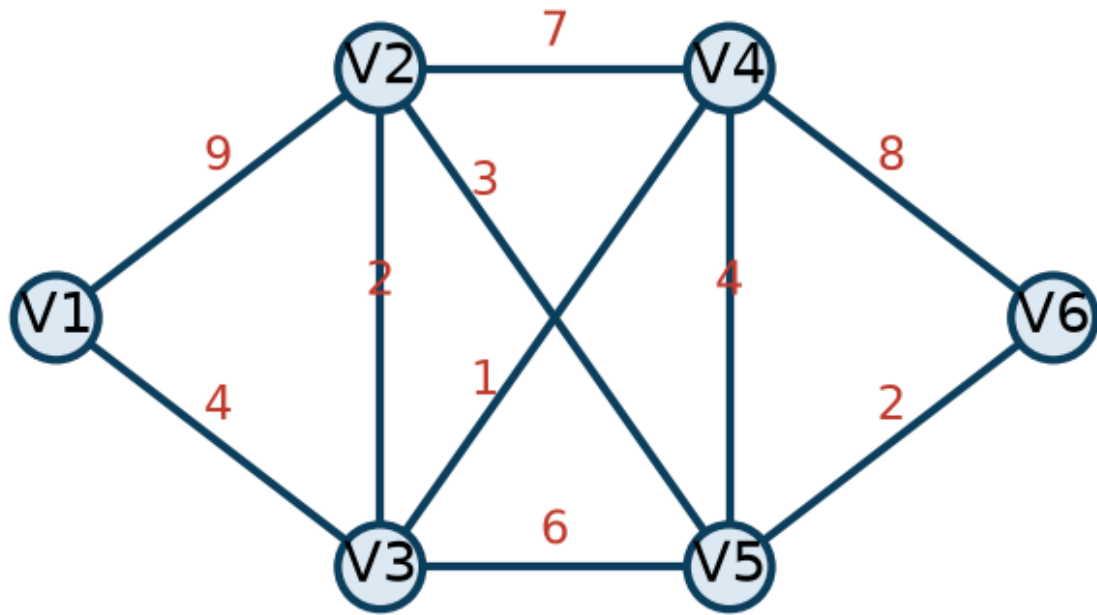
Distances: $b=1$, $d=2$, $c=4$, $e=5$.

Shortest distance $a \rightarrow e = 5$; path: $a \rightarrow b \rightarrow c \rightarrow e$

Q3. Use Dijkstra's algorithm to find the shortest path between the vertices V_1 and V_6 .

What is asked: the least-weight route from V_1 to V_6 .

Q5 graph



Q3 graph (edge labels are weights).

Run Dijkstra from V_1 .

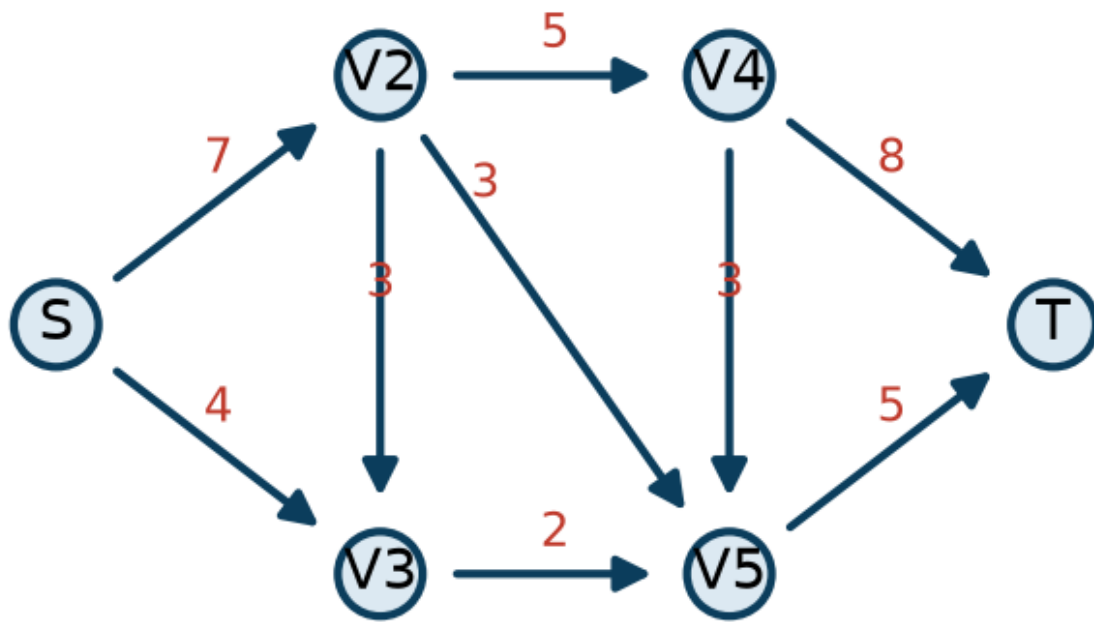
- $V_1=0$; neighbours: $V_2=9$, $V_3=4$.
- Settle V_3 (4): relax V_2 : $4+2=6 < 9 \rightarrow V_2=6$; V_4 : $4+1=5$; V_5 : $4+6=10$.
- Settle V_4 (5): relax V_5 : $5+4=9 < 10 \rightarrow V_5=9$; V_6 : $5+8=13$.
- Settle V_2 (6): V_5 via V_2 : $6+3=9$ (no change).
- Settle V_5 (9): relax V_6 : $9+2=11 < 13 \rightarrow V_6=11$.
- Settle V_6 (11).

Shortest distance $V_1 \rightarrow V_6 = 11$; path: $V_1 \rightarrow V_3 \rightarrow V_4 \rightarrow V_5 \rightarrow V_6$

Q4. Find the maximum flow in the network using the Ford-Fulkerson algorithm.

What is asked: the largest flow from S to T.

Q6 network



Q4 network — edge labels are capacities.

Method. Push flow along augmenting paths until none remains.

- $S \rightarrow V_2 \rightarrow V_4 \rightarrow T$: bottleneck 5 $\rightarrow$ push 5.
- $S \rightarrow V_2 \rightarrow V_5 \rightarrow T$: bottleneck 2 $\rightarrow$ push 2.
- $S \rightarrow V_3 \rightarrow V_5 \rightarrow T$: bottleneck 2 $\rightarrow$ push 2.

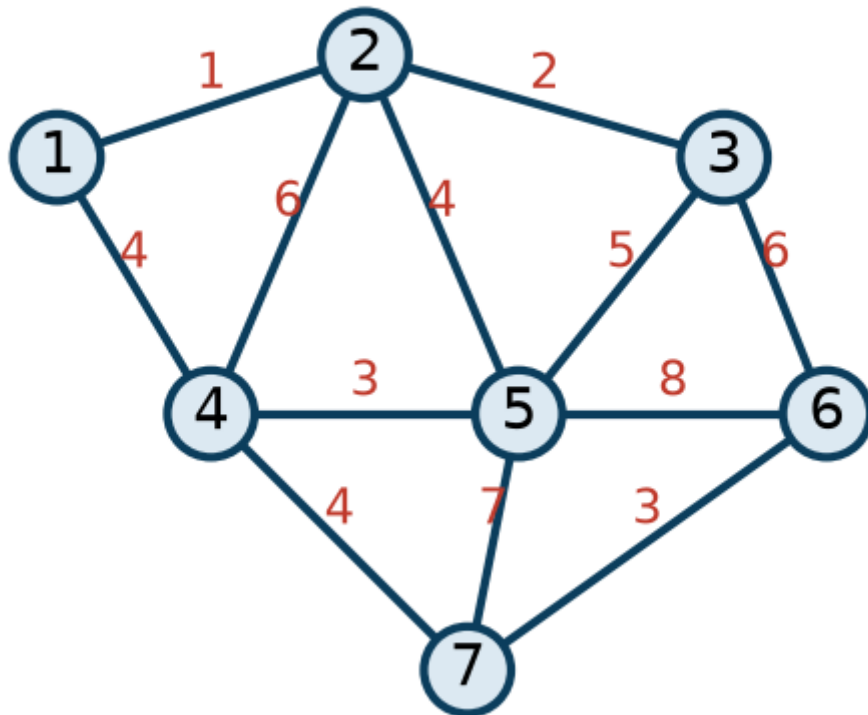
Total into T: $V_4 \rightarrow T = 4$, $V_5 \rightarrow T = 5$, giving $4+5=9$. No augmenting path remains.

Maximum flow = 9

Q5. Use Kruskal's algorithm to find a minimum spanning tree of the graph shown.

What is asked: the least-weight spanning tree.

Q7 graph



Q5 graph — 7 vertices; edge labels are weights.

Method (Kruskal). Sort edges by weight and take each one that does not form a cycle:

Weight	Edge	Take?
1	1-2	yes
2	2-3	yes
3	4-5	yes
3	6-7	yes
4	1-4	yes
4	2-5	no (cycle)
4	4-7	yes

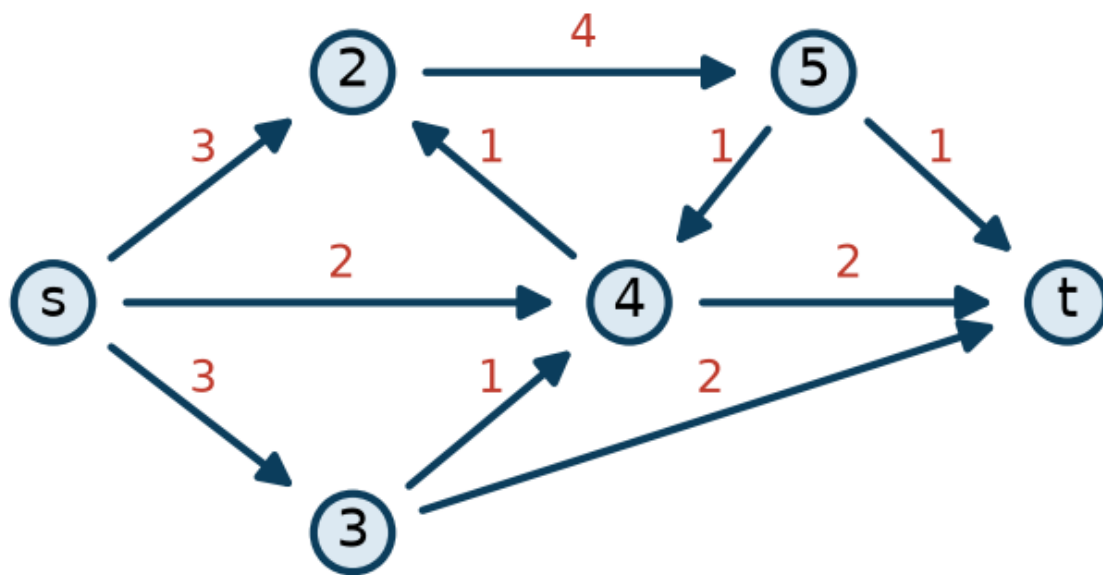
We now have $7-1=6$ edges: 1-2, 2-3, 4-5, 6-7, 1-4, 4-7.

MST weight = $1+2+3+3+4+4=17$

Q6. Find the maximal flow of the network using the Ford-Fulkerson algorithm.

What is asked: the largest flow from s to t.

Q8 network



Q6 network — edge labels are capacities.

Method. The only edges into t are $5 \rightarrow t$ (1), $4 \rightarrow t$ (2) and $3 \rightarrow t$ (2), so no flow can exceed $1+2+2=5$. We can achieve 5:

- $s \rightarrow 2 \rightarrow 5 \rightarrow t$: push 1 (limited by $5 \rightarrow t$).
- $s \rightarrow 4 \rightarrow t$: push 2.
- $s \rightarrow 3 \rightarrow t$: push 2.

Total = 5, and the cut $\{s, 2, 3, 4, 5\} \rightarrow \{t\}$ has capacity $1+2+2=5$.

Maximum flow = 5

17. Quick Revision Summary

Definitions to Remember

- **Tree:** connected graph with no cycle; equivalently, a unique path between every pair of vertices.
- **Rooted tree, level, height, parent/child, siblings, ancestor/descendant, internal/terminal vertex, subtree.**
- **Binary tree** (0, 1 or 2 children); **full binary tree** (0 or 2 children).
- **Spanning tree:** subgraph containing all vertices and no cycle.
- **Weighted graph; minimum spanning tree (MST).**
- **Flow network, capacity, flow, capacity constraint, flow conservation, value of a flow.**
- **Residual capacity/network, augmenting path, cut, minimum cut.**

Formulas & Results

Result	Statement
Edges in a tree	$n-1$ for n vertices
Full binary tree	k internal $\Rightarrow k+1$ leaves, $2k+1$ vertices
Binary tree height/leaves	$t \leq 2^h$ (i.e. $\log_2 t \leq h$)
Residual capacity	$c_f(u,v) = c(u,v) - f(u,v)$
Cut capacity	$\text{cap}(S,T) = \sum_{u \in S, v \in T} c(u,v)$
Max-Flow Min-Cut	$\max f = \min \text{cap}(S,T)$

Algorithms

Algorithm	Solves	Idea
DFS	Traversal / spanning tree	Go deep, use a stack
BFS	Traversal / spanning tree	Go level by level, use a queue
Kruskal	MST	Add cheapest edge that makes no cycle
Prim	MST	Grow one tree from a start vertex
Dijkstra	Shortest path (non-negative weights)	Settle nearest unsettled vertex, relax neighbours
Ford-Fulkerson	Max flow	Push flow along augmenting paths in the residual network

Key Points

- Both Kruskal and Prim give an MST (which need not be unique).
- Dijkstra needs non-negative weights and a connected graph.
- Ford-Fulkerson always terminates with the optimum; trial-and-error may not.
- Maximum flow value = capacity of the minimum cut (Max-Flow Min-Cut theorem).

— End of Unit IV (Trees and Networks) Notes —